Assignment 2

ADSA

Manny Asbanu
a1926920
Repository: https://github.com/mannyasbanu/ADSA

Description

For the task I wrote implementations in C++ for AVL tree insertion and deletion, and in-order, post-order, and pre-order traversal.

To support my implementations, I created a Node struct, Tree class, and helper functions for fetching node height, minimum node, right rotation, left rotation, vector printing, input processing, and updating tree root on insert and delete.

Each implementation was designed without the use of AI. Youtube videos and online resources were used for assistance with syntax and theory.

AVL Tree

The AVL tree is a self-balancing Binary Search Tree (BST). It is ordered such that the difference between the heights of left and right subtrees for any node is no more than one. Searching an AVL tree has time complexity O(log n) making it useful for frequent lookups. The balance of a subtree is calculated using the balance factor shown below.

$$\text{balance factor} = \text{left subtree height} - \text{right subtree height}$$

Nodes are inserted according to the rules of a normal BST. Inserting to the left of greater nodes and to the right of smaller ones. After insertion and deletion, the balance factor is recalculated. If the tree becomes unbalanced (balance factor > +1 or < -1) left and right rotations are performed. These operations are described below.

Left Rotation

The root node moves down to the left, and its right child becomes the new root. If the right child has a left child, it becomes the right child of the old root node.

Right Rotation

The root node moves down to the right, and its left child becomes the new root. If the left child has a right child, it becomes the left child of the root node.

The order of rotations to balance a tree after insert or delete operations are determined by the following four rules.

Left Left Case
A single right rotation is performed when the balance factor is greater than one and the node was inserted into the left subtree of the left child.

Right Right Case
A single left rotation is performed when the balance factor is less than negative one and the node was inserted into the right subtree of the right child.

Left Right Case
A left rotation is performed on the left child, followed by a right rotation on the root. This occurs when the balance factor is greater than one and the node was inserted into the right subtree of the left child.

Right Left Case
A right rotation is performed on the right child, followed by a left rotation on the root. This occurs when the balance factor is less than the negative one and the node was inserted into the left subtree of the right child.

Traversal

Three different Depth-First Search (DFS) algorithms are implemented to traverse each node of the AVL tree. These are pre-order, in-order and post-order. As they are DFS algorithms, they can be implemented using a stack or recursion. Each has a time complexity of O(n), as every node must be visited. The process of each traversal is described below.

Pre-order
Visit the nodes in order of the root node, left, then right. The general algorithm is to visit the root, traverse the left subtree, traverse the right subtree. Pre-order traversal is typically used to create a copy of the tree.

In-order
Visit the nodes in order of the leftmost node, root, then right. The general algorithm is to traverse the left subtree, visit the root, traverse the right subtree. For BST and thus AVL trees, in-order traversal gives nodes in non-decreasing order. It is typically used to evaluate arithmetic expressions in expression trees.

Post-order
Visit the nodes in order of the leftmost node, right, then root. The general algorithm is to traverse the left subtree, traverse the right subtree, visit the root. Post-order traversal is typically used to delete the tree.

Implementation

Below is an explanation of my implementations for each AVL tree operation and node traversal.

Insertion
Recursively traverse the tree starting from the root node, traversing left if the key is smaller than the root and right otherwise. On each recursive call, the root child is assigned to the result of the call. Once nullptr is reached, create a new node and return it, this assigns the inserted node as the child of the last valid node. While unwinding the recursive calls, the height of the root is set to the max of its two children, the updated subtree balance factor is fetched, and the subtree is rebalanced according to the four rules. The root is returned at the end. After the root of the tree is reached, the height of each node has been updated and rebalanced.

Delete
Recursively search the tree starting from the root node, searching left if the key is smaller than the root and right otherwise. On each recursive call, the root child is assigned to the result of the call. Once the key is found, the node is deleted and replaced according to three rules. If it has no left child, the right child replaces it. If it has no right child but has a left child, the left child replaces it. Otherwise, it is replaced by its in-order successor, which is the smallest node greater than the current. While unwinding the recursive calls, the height of the root is set to the max of its two children, the updated subtree balance factor is fetched, and the subtree is rebalanced according to the four rules. The root is returned at the end. After the root of the tree is reached, the height of each node has been updated and rebalanced.

Pre-order
Push the root onto the stack. While the stack is not empty, pop the top node and visit it. Push its right and left children onto the stack.

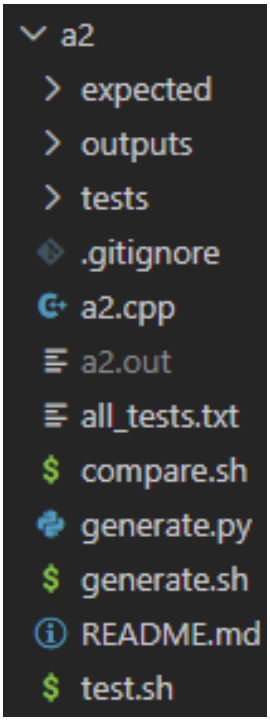
In-order
Set the current node to the root. While the stack is not empty or the current node is not equal to nullptr, perform the following. Push the current node onto the stack and set the current to its left child. Once the current node is equal to nullptr, visit its parent which is on the top of the stack. Set the new current to the right child of the parent.

Post-order
Set the current node to the root. While the stack is not empty or the current node is not equal to nullptr, perform the following. Push the current node onto the stack and set the current to its left child. Once the current node is equal to nullptr, access the node at the top of the stack. If it has a right child, set it to current. Otherwise, visit the node and pop it off the stack.

Testing and Debugging

My testing pipeline was created using the Codex CLI coding agent from ChatGPT. I prompted the agent using the context of my previous testing workflow implemented in assignment 1 as well as the input and output descriptions. By using the previous workflow as a reference and providing access to my file structure, the exact testing steps remained identical to last time. The following is the exact prompt provided to Codex. The image on the right is the folder structure of my testing workflow.

```
I need to develop a testing workflow for my assignment 2 a2.
The assignment was to develop AVL tree insertion deletion
and different traversal methods. The input output format is
as follows:
Sample input 1
A1 A2 A3 IN
Sample output 1
1 2 3
Sample input 2
A1 A2 A3 PRE
Sample output 2
2 1 3
Sample input 3
A1 D1 POST
Sample output 3
EMPTY
I need you to develop a clear testing pipeline. For
reference, please follow the structure of my previous
assignments testing workflow a1. My previous assignment
testing workflow is already implemented in the a1 folder, I
need to implement a new one for my new assignment in a2.
```

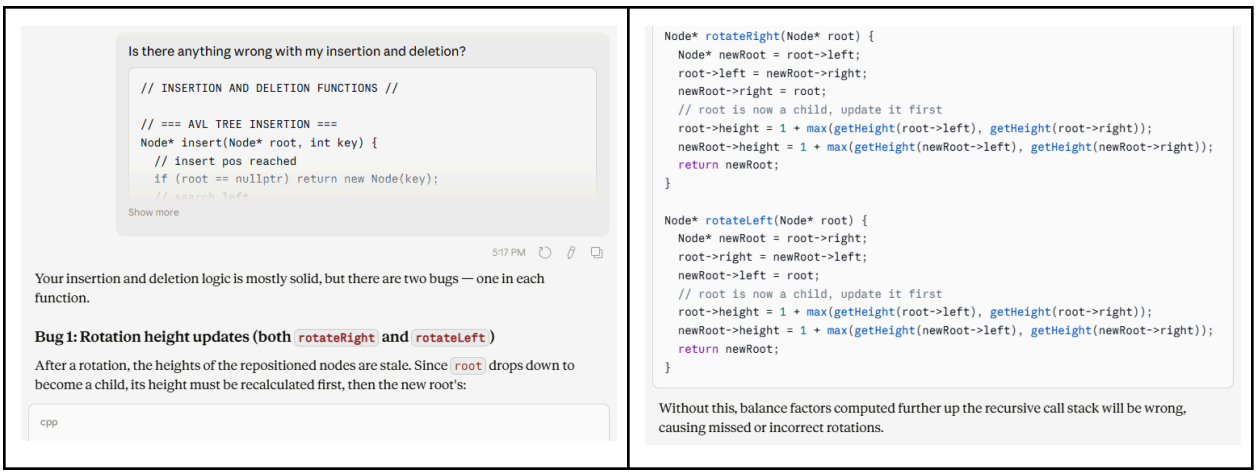


The workflow and file structure is as follows. All test inputs are generated or custom added to a single text file all_tests.txt. The bash file generate.sh converts the text file into individual test case files t<num>.txt stored under the tests folder. A python file generate.py runs on each individual test case, returning each expected result t<num>.txt under the expected folder. The bash file test.sh passes each test case into my compiled a2.out program file, outputting the results t<num>.txt to the outputs folder. Finally, compare.sh compares each text file inside the outputs folder to the expected folder. Returning any differences found and the associated file number.

A limitation and possible risk I identified during my testing process, is the python file used to generate expected results for each test case uses an implementation generated by Codex instead of an external library. If I were to improve the validity of my testing, I would try to source an existing implementation online, or combine library functions for each solution component (e.g. AVL tree insertion, deletion and traversal). I did not prioritise this as after manually verifying

results generated by the Codex implementation using an online AVL tree visualiser I could not locate any errors.

The only error my implementation produced occurred on 5 of the 50 test cases. My debugging process was as follows. I input the test case into the online AVL tree visualiser https://www.cs.usfca.edu/~galles/visualization/AVLtree.html, and manually verified the traversal through observation. After confirming the error was my algorithm and not the Codex implementation, I passed my traversal function into Claude. It could not locate any errors. After passing my insertion and deletion functions it was clear. I had forgotten to update the height of the nodes inside the left and right rotate functions. After performing a rotation, I had to re-calculate the height of the old root and its child which became the new root. The prompt provided to Claude and its output is shown below.



Is there anything wrong with my insertion and deletion?

```
// INSERTION AND DELETION FUNCTIONS //

// === AVL TREE INSERTION ===
Node* insert(Node* root, int key) {
  // insert pos reached
  if (root == nullptr) return new Node(key);
  // search left
```

Show more

5:17 PM

Your insertion and deletion logic is mostly solid, but there are two bugs — one in each function.

Bug 1: Rotation height updates (both `rotateRight` and `rotateLeft`)

After a rotation, the heights of the repositioned nodes are stale. Since `root` drops down to become a child, its height must be recalculated first, then the new root's:

cpp

```
Node* rotateRight(Node* root) {
  Node* newRoot = root->left;
  root->left = newRoot->right;
  newRoot->right = root;
  // root is now a child, update it first
  root->height = 1 + max(getHeight(root->left), getHeight(root->right));
  newRoot->height = 1 + max(getHeight(newRoot->left), getHeight(newRoot->right));
  return newRoot;
}

Node* rotateLeft(Node* root) {
  Node* newRoot = root->right;
  root->right = newRoot->left;
  newRoot->left = root;
  // root is now a child, update it first
  root->height = 1 + max(getHeight(root->left), getHeight(root->right));
  newRoot->height = 1 + max(getHeight(newRoot->left), getHeight(newRoot->right));
  return newRoot;
}
```

Without this, balance factors computed further up the recursive call stack will be wrong, causing missed or incorrect rotations.